A3-PS022-TC181

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Category: Cybersecurity

PacketSentinel — Monitoring and filtering malicious traffic

Krishnaja Jinka

Krish30.j@gmail.com

AIORI Hackathon 3

October 13, 2026

Abstract & Problem Description

This project builds an XDP filter on a Linux host pair that performs bounded UDP DNS parsing and applies per-source and global rate limits during three synthetic DNS query flood patterns. Its performance is compared against a no-filter baseline and an nftables baseline under identical conditions, measuring throughput, CPU usage, legitimate-query success and packet drops.

Problem Description

- High-volume UDP DNS query floods saturate Linux servers: the kernel spends CPU allocating socket buffers for packets before they reach user space.
- Legitimate DNS queries get lost in the flood, reducing successful resolution.
- Filtering after socket allocation spends the same resources the flood is trying to exhaust.
- A bounded, low-cost, measurable per-source and global rate limiter is needed.

Background

- eBPF lets small, verified programs run inside the Linux kernel. XDP attaches them to the network driver's receive path.
- An XDP program returns a verdict (pass or drop) before the kernel allocates socket buffers for the packet. This is where the CPU saving comes from.
- eBPF maps store shared state such as per-source counters and token-bucket values.
- A token bucket gives each source a budget that refills at a fixed rate. Packets beyond the budget are dropped.
- DNS messages (RFC 1035) start with a fixed 12-byte header. Bounded parsing reads only the header and first question, with a length check at every step.
- XDP runs in native mode when the NIC driver supports it, or in generic mode otherwise. Generic mode runs after socket-buffer allocation, so savings are smaller; the setup must be documented.

Literature & RFC Review

- Linux kernel BPF documentation: defines the eBPF instruction set, XDP program types and map types used in this project [1].
- RFC 1035: defines the DNS message format. It is the basis for the bounded header and question parsing [2].
- nftables wiki: documents the netfilter-based firewall, including its rate-limit and meter expressions. It serves as the practical baseline [3].
- Only the most relevant sources are listed here. Full DNS semantic parsing, universal PRSD detection and reflection-attack mitigation are out of scope and are not reviewed.

Proposed Solution

- An XDP program on the receiving host parses Ethernet, IPv4 and UDP headers with bounds checks.
- Packets to UDP port 53 get a bounded DNS header and first-question check.
- A per-source token bucket lives in an eBPF hash map keyed by source IP.
- A global token bucket lives in a single-entry array map.
- Conforming packets pass to the kernel. Excess packets are dropped at XDP.
- A userspace loader reads map counters for drops and passes.
- A traffic harness generates three synthetic DNS flood patterns plus legitimate queries.

Pseudocode (prototype)

```
on_packet(ctx):
  if not parse_eth_ip_udp_bounded(ctx): return PASS
  if udp.dst_port != 53:                return PASS
  if not dns_header_bounded(ctx):
      count(MALFORMED); return DROP
  b = src_map[src_ip] or new_bucket()
  refill(b, now); refill(global, now)
  if not take(b) or not take(global):
      count(DROP); return DROP
  count(PASS); return PASS
```

Optimization Proposed

- Earlier drop point: packets are discarded before socket-buffer allocation.
- Bounded parsing: only fixed-size headers and a capped name length are read.
- Identical traffic and measurement for all three configurations (no filter, nftables, XDP).

Aspect	No-filter baseline	nftables baseline	XDP proposal
Drop point	Socket / application	Netfilter hook in kernel stack	Driver (native or generic XDP)
Per-source limit	None	Rate-limit rule	eBPF per-source token bucket
Global limit	None	Rate-limit rule	Global token bucket
Packet parsing	None	Rule matching	Bounded UDP/DNS header parsing
Measured	CPU, throughput, drops, legit success	Same metrics	Same metrics

Measured values to be filled in after lab runs. No performance figures are claimed here.

Solution Architecture & Design

Synthetic DNS traffic path on the Linux host pair

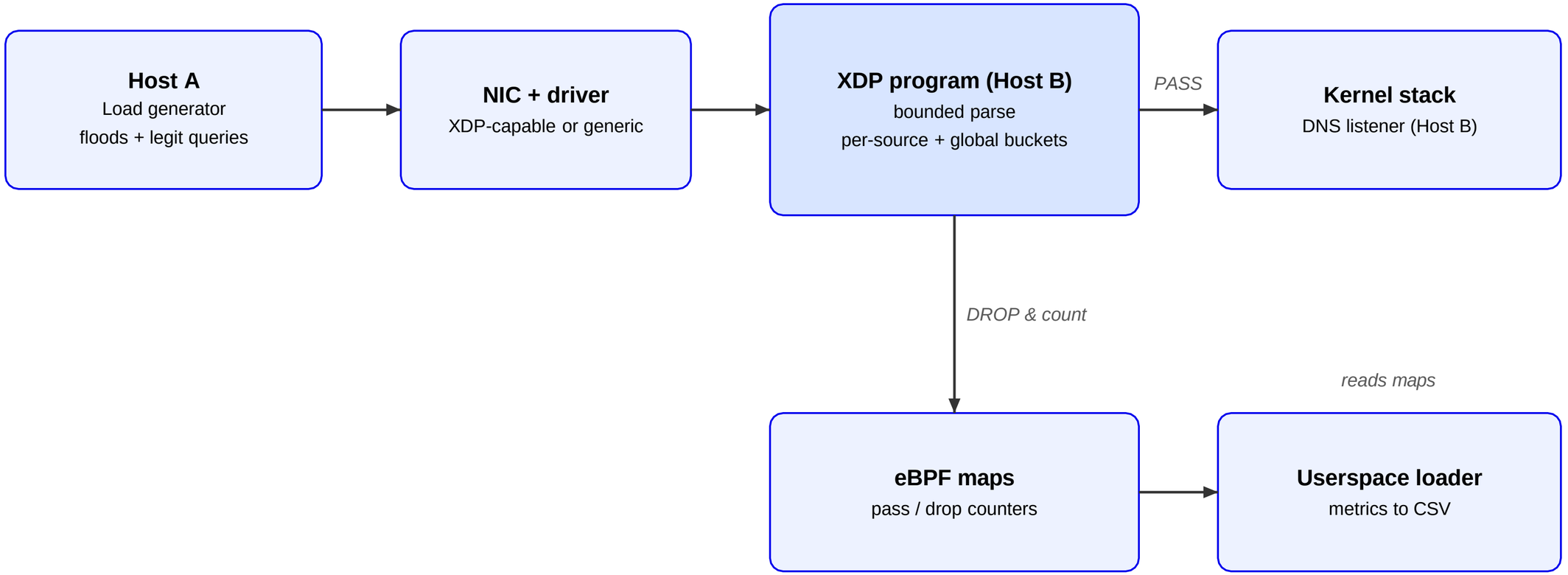


Figure: System architecture (XDP filter on the receiving host)

Data Flow Diagram

Packet path from generation to verdict

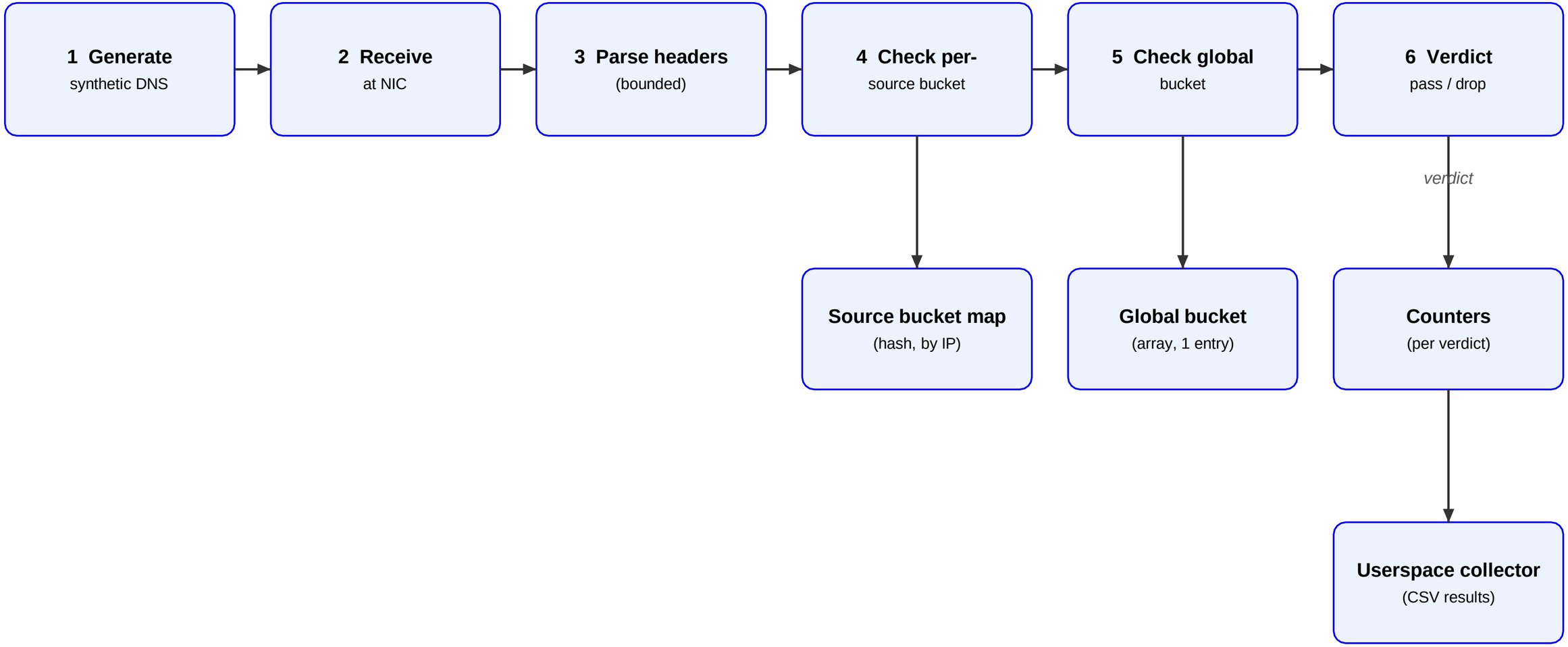
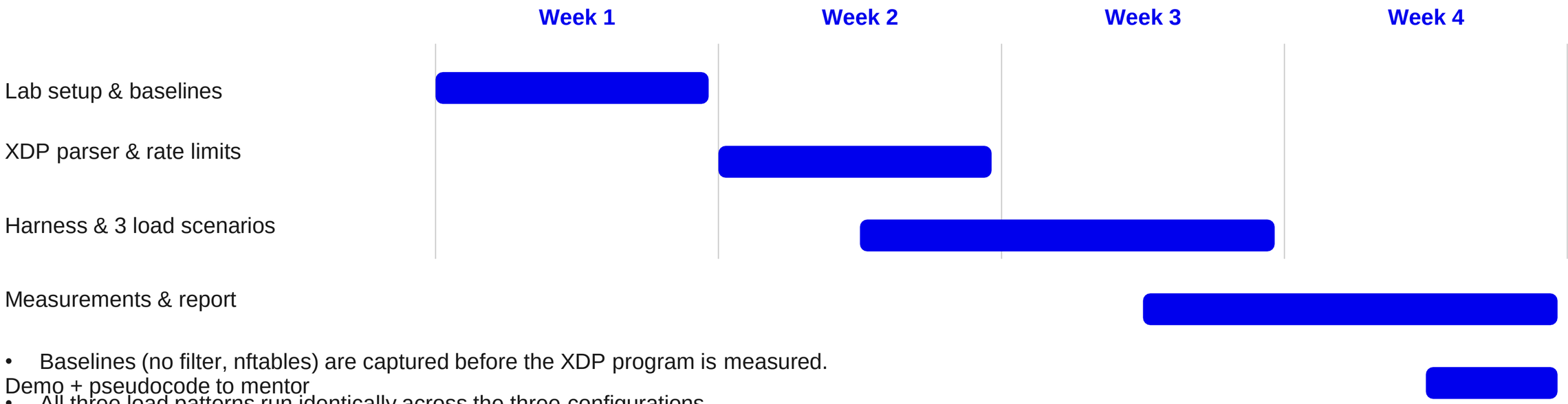


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Indicative four-week plan; adjust to the hackathon schedule



- Baselines (no filter, nftables) are captured before the XDP program is measured.
- All three load patterns run identically across the three configurations.
- Deliverables: private GitHub repository named after the team, aiori-hackathon added as collaborator and given ownership, PDF uploaded in the required structure, and a prototype demo with pseudocode for the mentor.

Bibliography

- [1] Linux kernel documentation, "BPF documentation," docs.kernel.org/bpf/.
- [2] P. Mockapetris, "Domain Names - Implementation and Specification," RFC 1035, IETF, Nov. 1987.
- [3] nftables project, "nftables wiki," wiki.nftables.org.

Thank You for Your Attention!